Schemas 101 notes

Read the key ideas about schemas and the questions to ask.

A schema is an agreement, not a technology

County leaders already work with schemas. Every standardized application form, budget code, or reporting template contains an agreement about what a piece of information means. A **schema** makes that agreement explicit so that people and software do not have to guess.

Imagine a request for a building inspection. Staff record the request number, address, request date, inspection type, and current status. A useful schema describes those pieces of information: what each field means, which values belong in it, and which pieces connect to other records.

The schema is **not** the actual inspection, a new computer system, or a requirement to learn programming. It is the set of rules that keeps an ordinary fact from becoming a different fact in the next office.

- **Names:** What do we call each piece of information?
- **Meanings:** What exactly counts as that information?
- **Rules:** What values and relationships are allowed, and how do they stay consistent?

A schema can be simple. A short, clearly maintained description of fields can be more useful than a sophisticated document nobody understands.

Leadership question: *Does this application have a documented, stable schema that another team can understand?*

You do not need to interpret every technical detail. You need confidence that the people operating and connecting your systems can rely on the same agreement.

The same label can mean two different things

Two departments send leadership a report with a column labeled **People Served**. One department counts each person once during the year. The other counts every visit, including repeat visits by the same person.

Both reports may be internally accurate. Together, they can produce an inaccurate conclusion.

That is a schema problem: the label matches, but the definition does not. The reverse also happens. One system says **Request Number**, another says **Service ID**, and both might be referring to the same underlying request.

A spreadsheet export does not solve this. A spreadsheet moves values. It does not automatically explain what those values mean, which definitions were used, or whether a record in one system refers to the same record in another.

When staff say, "We will clean that up in Excel," ask *why*. Are they fixing a few data-entry mistakes, or repeatedly translating the same mismatched definitions between two systems?

Repeated manual translation is not just an inconvenience. It makes reporting slower and less reliable, and it can hide problems that surface only after a budget or program decision.

Leadership question: *When two systems report this field, are they describing the same thing? How have we verified that?*

One work order, two systems

Consider a routine County facilities repair. A work-order application tracks the problem, the crew, and whether the repair is complete. A finance application tracks the purchase, invoice, and payment.

Leadership wants one answer: **How much did we spend completing these repairs?**

Both applications can export files. That does not mean they can automatically produce one reliable answer.

Facilities application	Finance application	Leadership concern
Work order number	Invoice reference	Do both values point to the same job?
Completed	Paid	Is the work finished, or has an invoice been paid?
Work completion date	Payment date	Which date belongs in this report?

The two systems do not have to use identical names. They do need **a documented way to connect the right records and preserve the right meanings**.

Staff might build a mapping, which is simply an agreed translation between two sets of fields. They should demonstrate it with real-looking test records, including a missing invoice and a job with several invoices.

Leadership question: *Can you show me how the schemas of these applications fit together for this work order?*

A useful demonstration ends with matching records and a clear explanation of what does **not** match. A vague assurance that "the systems integrate" is not evidence by itself.

Stable does not mean frozen

Software changes. Programs change. Reporting requirements change. A schema that could never change would become a liability.

Stable means predictable, documented, and managed—not frozen forever.

Imagine that an application currently has only two request statuses: **Open** and **Closed**. The department adds **Waiting for Customer**. The change may be sensible, but a dashboard that assumes every record is either Open or Closed could now give a misleading result.

This is where discipline matters. Before a new status appears in production, teams should know:

- what the new status means;
- which reports, forms, and connected applications rely on the old values;
- how existing records will be handled;
- who will test the change; and
- how users and vendors will be informed.

A *version* is simply a way to label which set of rules applies. A *change notice* tells other teams what has changed. These are routine management tools, not exotic engineering rituals.

Leadership question: *Is this schema stable and versioned? Who tells other systems when it changes?*

Be wary of "We change fields whenever the vendor needs to." The real concern is not change; it is change without coordination.

A record needs a reliable identity

Names are useful to people. **Identifiers** are what let systems reliably distinguish one record from another.

The facilities team may call a repair **WO-2047**. The finance team may call its payment **PAY-990**. Those values are not wrong. But a system cannot safely assume they refer to the same job just because both records mention "Main Street."

Someone needs to define the connection. Sometimes both systems can carry the same stable work-order number. Sometimes there must be an approved lookup linking one system's ID to the other's. Either approach can work when it is explicit and tested.

The same problem becomes more serious with people, services, and other sensitive records. Two records with similar names are not automatically the same person. Matching must use appropriately authorized methods and data—not a guess based on a name or address.

- Which field uniquely identifies a record in this application?
- Does that identifier remain the same when other fields are updated?
- How does the next system recognize the corresponding record?
- What happens when the match is missing, duplicated, or uncertain?

Leadership question: *Do these systems have reliable identifiers, and can staff show how they match?*

An ID is not a promise of accuracy, but it gives teams a dependable starting point for reconciliation.

The business must own the meaning

A technology team can tell you where a field lives. It cannot decide alone what **Successfully Referred**, **Case Closed**, or **Service Delivered** should mean for a program.

Those are business decisions. Program staff understand the work, legal and privacy specialists understand constraints, and technology staff understand how systems represent and exchange information. Schema governance needs all of them.

That does **not** mean every County application must use one enormous, identical schema. Departments have different missions. The goal is consistency where information needs to connect—and honest, documented differences where it does not.

A lightweight shared definition should answer:

- What is this field supposed to describe?
- What values may appear in it?
- Who is responsible for deciding when the meaning changes?
- Which applications or reports rely on it?

When nobody can name the person or group responsible for a definition, disagreements tend to persist until they become expensive integration problems.

Leadership question: *Who is the business steward for these definitions, and who approves changes that affect other departments?*

Departments do not have to agree on everything. They do need an agreed way to resolve the definitions that affect shared work.

A shared schema is not permission to share everything

If two systems can understand each other's data, that does **not** mean every field should move between them.

The schema describes what information exists and how it is organized. It does not grant permission to disclose protected information. A transfer still needs an approved purpose, appropriate legal and policy review, access controls, and limits on what is necessary.

Consider a referral handoff. The receiving program may need to know that a referral was accepted, declined, or still pending. It may not need the complete case history that produced that status.

A good integration design asks which facts the receiving team actually needs. It should not default to copying the entire record simply because that is technically easier.

- **Can these schemas communicate?** That addresses shared meaning and dependable exchange.
- **Should these facts be shared?** That addresses purpose, privacy, security, authorization, and retention.

The answer to the first can be yes while the answer to the second is no—or yes only for a limited set of fields.

Leadership question: *Which fields are necessary for this handoff, and who has approved access to them?*

Schema compatibility and good stewardship reinforce each other. Neither replaces the other.

Do not buy a promise of integration

During a software demonstration, "We have an API" or "We export to Excel" can sound reassuring. Those may be useful features. Neither proves that the information matches what another County application expects.

A better conversation names the other application and the County task. For example: **Show how this proposed work-order system would exchange job identifiers, status, dates, and costs with our finance system.**

Ask for a practical demonstration or documented example, not a slide declaring a product "integrated."

1. Can you provide documented field definitions and allowed values?
2. What identifiers remain stable when records change?
3. Which fields map to our named existing system, and which do not?
4. How are corrections and updates handled after the first transfer?
5. How are schema changes announced, versioned, and tested?
6. Can we export our records and their definitions in a usable format if we change systems?
7. Which integration work, maintenance, and changes carry additional costs?

A vendor may reasonably need time to produce this evidence. The important thing is a clear, reviewable answer before the County depends on the connection.

Leadership question: *Does this integrate well with the schema of our other application? Show us how you know.*

These are evaluation questions, not a substitute for County procurement, security, or contracting requirements.

The 60-second schema check

You do not have to become a database expert. You can make schema questions normal in meetings.

Ask these seven questions when a project depends on information moving between systems:

1. **Is there a documented, stable schema?** Can staff explain it in ordinary language?
2. **Do the fields mean the same thing?** Where do definitions differ?
3. **How do records match?** Which identifiers carry through the handoff?
4. **Does it fit our other application?** Has someone demonstrated the mapping with a named system?
5. **Who manages change?** Who documents, tests, and communicates new fields and values?
6. **Is the exchange appropriate?** Which data is necessary, authorized, and protected?

7. **Can we take our information with us?** Are exports and field definitions available if a vendor changes?

You are not asking the team to recite code. You are asking it to show a dependable agreement.

> *Does this have a stable schema?*
>
> *Does this integrate well with the schema of our other application?*

Follow with **"Can someone show us a real example?"**

A good answer may uncover work that still needs doing. That is progress, not failure. Discovering the mismatch before a rollout is cheaper than explaining it to the Board after inaccurate numbers are published.

Start with one handoff, not a Countywide rewrite

A County does not need to replace every application to become better at schemas. Start with a real handoff that already matters.

Choose a contained example, such as a facilities work order moving from service completion to payment reporting. Include the program owner, the receiving team, technology staff, and the right privacy or security reviewers.

1. **Pick one decision.** What is the business question this connection should answer?

2. **Choose two systems.** Name the sending application and the receiving application.

3. **Compare a few fields.** Start with record ID, status, and one meaningful date or amount.

4. **Test sample records.** Include a normal example, a missing value, and a record that changes after initial entry. Use approved test data.

5. **Write down the mismatches.** Which meanings, identifiers, or allowed values differ?

6. **Agree on ownership.** Who maintains the translation and is notified when either schema changes?

7. **Repeat the test.** Confirm that the handoff remains understandable after a change.

Success is not "a consultant built a connection." Success is that staff can explain the information flow, spot an exception, and keep it working without depending on undocumented knowledge.

Leadership question: *What one cross-system handoff can we demonstrate before promising a larger integration?*

What to say at the next meeting

Schema discussions should feel as ordinary as asking for the budget, timeline, or staffing plan.

Instead of asking...	Ask...

"Does the product integrate?"	"Can you show how its fields and statuses match our existing application?"
"Can we get a spreadsheet?"	"Will the export include understandable, documented definitions and stable IDs?"
"Will this report be accurate?"	"Are we counting the same thing in both systems?"
"Can IT fix that later?"	"Who owns the definitions and tests changes before the connection breaks?"

A responsible team may answer, **"Not yet—we found a mismatch."** That is useful information. It gives leadership a real choice about cost, scope, and timing. Do not punish the team for surfacing it.

At the end of a purchasing, reporting, or integration discussion, ask for three plain-language statements:

- **We agree what the important fields mean.**
- **We know how the two systems match records and handle changes.**
- **We know who maintains the agreement and what information is permitted to move.**

If those statements cannot yet be supported, the project may still be worthwhile. It is simply not ready to be described as a proven integration.

The takeaway: You do not need to speak in acronyms. You need to make "schema" a comfortable word for a very practical question: *Are we talking about the same thing?*